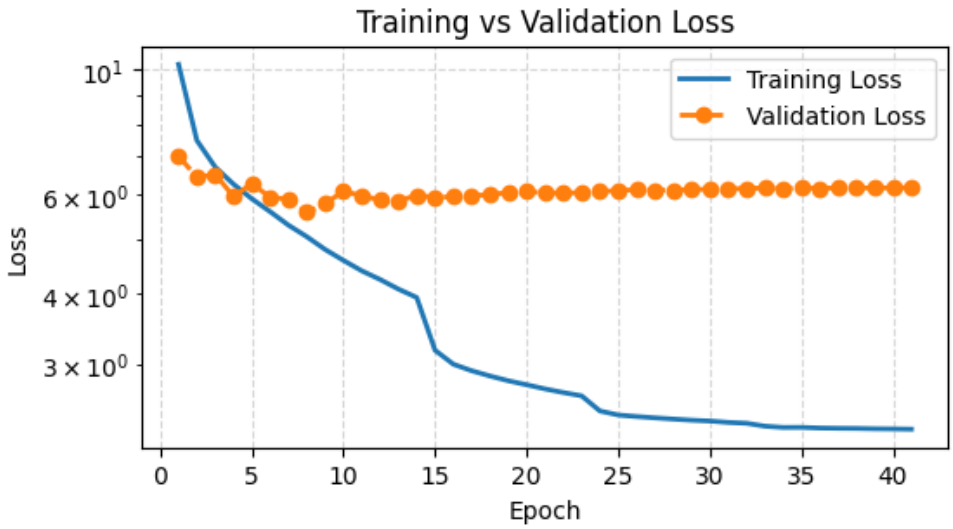
ML to improve streamflow forecasting

Improvements using a new model

Draft



hydrique.

Hydrique Ingénieurs
Ch. du Rionzi 54 bis, CH-1052 Le Mont-sur-Lausanne
+41 21 784 33 13
www.hydrique.ch info@hydrique.ch

TABLE DES MATIÈRES

1 Introduction

1.1 Context

Objectives received

Hydrique engineers is currently using a MLP model to improve their streamflow forecast coming out of their software RS. Their MLP has already been optimized multiple times and a score over a +24h horizon has achieved around MAPE=10.

A package with a promising variety of models (NeuralHydrology) is to be tested, to check if better results can be achieved by one of their models. The primary goal is thus to download and become familiar with this package, to then test their Auto-Regressive Long-Short Term Memory (AR-LSTM) model.

1.1.1 Old way: MLP workflow

Model

Short description to how they were using the Multi-Layer Perceptron (MLP), what was tested and maybe the limitations of the previous model:

Limitation: for example, to keep track of the time and have long series of time as inputs, it becomes then a too large vector as inputs and has no idea of the "chronology" of our data.

Input

As input, the model took only the last 24 hours of measures to predict all horizons (To ask)

Horizon of the prediction & computation of the prediction

The targeted horizons of prediction were of +24, 46, 72 and 96 hours. Only the last element of this prediction was computed and the model was trained to gives and be accurate only for this point. This leads to a major problem, to do an accurate forecast, we had to use a backward rolling window, predicting +1 to +24 hours individually, only the +24h having access to the latest (current) measure.

The MAPE score obtained with the MLP model were:

- 24h – training: 9.7 – validation: 9.8
- 48h – training: 12.1 – validation: 12.16
- 72h – training: 12.8 – validation: 12.57
- 96h – training: 13.9 – validation: 13.2

Remark: These results were obtained for a (relatively short) training and validation period over 2 and 1 years respectively.
Training period: 1.05.2020- 31.10.2022.
validation period: 1.05.2023 – 31.10.2023

1.1.2 Constraints

None 😊

Tools: Package NeuralHydrology, Knowledge about the App Massa, the way Hydrique and RS work, my code, analysis and ML model construction and training skills.

Colleagues: technically Xavier and Josep, still waiting on them to answer my questions on the GitHub issue.

Given the low answer rate and the multiple uncertainties, most of my time was passed trying and solving one problem or questioning at the time.

Not very enjoying having both several ideas and leads of improvements and not knowing if it was ok with the way the company work or was already tested by Josep.

1.2 Leads to improve our score – Axes of improvement

To possibly improve our forecast, we can modify these specific axes:

1. Model:
   Which model do we want to use, why and what are the expectations using such model.

2. Once the model is selected, the data can be extracted, organized, and cleaned (pre-processing is essential in Machine Learning).

3. Choice of hyperparameters and training options

Text left

Text right

1.3

2 Technical information

Part where a technical background is built to have some basic knowledge to better understand the results and motivate the choice of models.

2.1 Data acquisition:

Concerning the dataset receive from Josep, i used it in the beginning and afterward, having too many features unused for my case, created one personal dataset. I verified some of the used features and it seemed in accordance with the BD_Backup data on the disk X.

For my personal dataset ('massa_2'), i kept a minimal number of features, chosen myself, and added the meteorological forecasts associated with my selected stations:

- Temperature: Binn, Visp, Eggishorn, Jungfraujoch
- Precipitation: Bruchji, Visp, Fieschertal
- Radiation: Eggishorn, Visp, Jungfraujoch
- Time: hr(=1 at noon and 0 at 0000), hr_sin (sine with hours of a day), daylight(peak during June, sunny hour ratio in a day), doy_sin (sine with day of year)

NOTE: all features are not used in each model, a combination of them is tested. (a lot of combination has been tested, but it depends strongly on the model and hyperparameters chosen, from what has been observed)

1. Any feedback, advice or list of interesting stations is welcome
2. In addition to these data, a basic Jupyter notebook can be used to add rolling features (sum or average over a given window of previous points).

2.1.1 Questions:

- which inputs from RS would we want to implement (or at least test them). Heard from Luca and Johann that it improved the MLP model by a large factor -> sounds interesting and should be explored.
  Once the list is set, the question of the creation and availability (location) of such data is to be discussed.
- Period available for training: For now, my meteorological forecasts are from ICON CH1, meaning the first data available is from the 1st March 2020 at 0000. That shortens greatly our training period. Is there a way to ask for newly created forecasts using the chosen model (that can be changed if a better one is proposed) for a past period?

I think that increasing our training period (that is of around 3 years now) could be an "easy" way to improve the efficiency of the model.

Data Preprocessing:

- Was this question already discussed? For example, the choice of the scaler (min-max or centre with mean/var), the seasonality from our data, etc.
  It is possible that removing the seasonality of our data and target would allow the model to better react to a variation from a "classical value given this time of a year."

2.2 Why try LSTM models?

Introduction of LSTM models

Long-Short Term Memory models is a type of Recurrent Neural Network (RNN), that are models designed to process sequential data with recurrent connections. They, to cite one of their properties, allow to retain information from their previous Inputs or Outputs.

Pros of LSTM with respect to RNN for TS forecasting

LSTM architectures are imposing themselves in the domain of Time series manipulation among other RNN mainly by their capability of solving the problem of exploding or vanishing gradients when training with long time-series, of the classical RNN architecture. Indeed, the problem of the gradient hinder the model to correctly learn and causes the training to be inefficient.

2.2.1 LSTM description – Architecture analysis?

2.2.2 Typical uses, examples from papers

- Natural Language Processing: Machine translation, text generation, sentiment analysis, and speech recognition.
- Time Series Analysis: Stock market prediction, weather forecasting, and signal processing.
- Other tasks: Handwriting recognition, image captioning, and video analysis.

Sources and examples

A large variety of interesting implementations and presentations of LSTM models is available on the internet, some of them are gathered in the reference section [reference to the section]. An

availability

interesting example of implementation of LSTM models is presented here: https://machinelearningmastery.com/how-to-develop-lstm-models-for-time-series-forecasting/.

Choice of the model

The diversity of possible models, all with their own characteristics, motivate even more the work that must be done in advance of the development phase to choose wisely a model to test.

In the following, a short explanation of each model judged “interesting” by myself is presented.

2.3 How each step is done with NH

ggg

3 AR-LSTM

I was first asked to interest myself with this model in the NH package because it could have some practical properties for our streamflow forecast capability and usage.

Sources:
[https://www.tensorflow.org/tutorials/structured_data/time_series#advanced_autoregressive_model,]

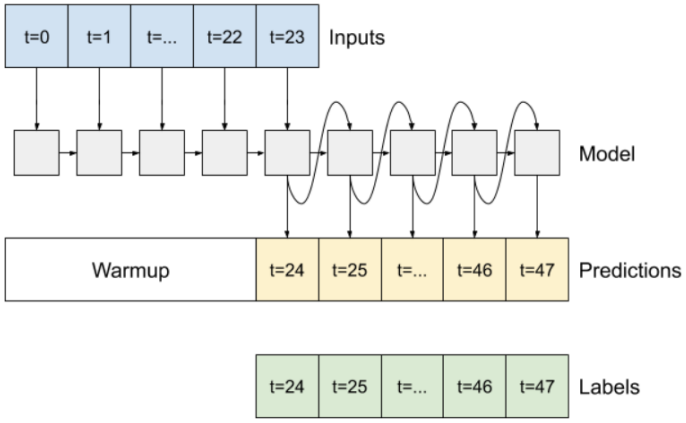


Fig. 1: Diagram of a typical Auto-Regressive LSTM Model
https://www.tensorflow.org/tutorials/structured_data/time_series#advanced_autoregressive_model

Description:

This model's interesting feature lies in the fact that it should be able to produce output(s) even in the case where the input is not anymore available.

As a concrete case, it should be able to work on a sequence of measure features (context window, takes measures of essential data – precipitation, temperature, radiation for example) producing an output at each time step (that could be compared to actual measure of streamflow in this window). The second phase would then consist on feeding it only its previous outcome on a sequence of a given length (our wanted horizon of prediction).

Pros:

- Large quantity of measured streamflow and meteo data
  -> no need of meteo forecasts (that are not available from a long time)

- Architecture relatively easy to understand and construction of sequences straightforward

- Would use all measures until time t to produce the predictions +1 to +24h, not only the +24h time step (as implemented by the MLP).

Cons:

- No new inputs during forecast phase?
- If the model introduces a bias on its output, this bias is multiplied by the number of point we want to predict (could be small for an +24h prediction, but not anymore on the +96h prediction)

POV:

- The idea of feeding the model its own output and producing all points of an horizon of prediction is a good point.
- A correction of the bias could be possible with a model trained to correct the model outputs to bring it near the measures again?

Concerns – questions:

Following this description and how the NH repo is constructed (mainly the sequence construction), I need to clarify some points.

- Choice of features to train, repartition Measures-Forecasts:
  - Only use measure on a sequence of L points and then let the model predict let us say 24 points using only its previous output?
  - Should the model have access to meteorological forecast during the AR-part?
- Having only its previous prediction, it seems unrealistic to have a good score after 10 predicted points, without some feedback to fix its derive or bias.
  - Once again, in the idea, feeding the model its last output to let it learn when he is good or not is a great idea, but we would have to build entirely this model, and maybe create our own sequencing too. It must be implemented in a logical way and in a way its usable in the “operational” workflow.

Results:

In short, one model was trained at first by me when I was not entirely understanding the package and the implementation of this model. Note that I cannot say I understood entirely the way the model was implemented in NH and how it should be used by some random user of the NH package. Anyways, without having an idea of what would we like to implement as Hydrique, I am not able to decide whether the already implemented model could be useful to us.

(The first model was achieving a good score (MAPE~7) but with some cheating unintentionally induced. This score is then not relevant. As previously described, the current score magnitude of my other model is around 11 to 12.)

4 Hindcast-Forecast LSTM

Context

Due to the limitations and concerns I had trying to build the AR model, I went through the other models from the NH repo. Two of them sounded very promising, solving the problem of our sequence being composed of the same features for all data points. The two models are the “Sequential LSTM” and the “Handoff LSTM” models, both having the same base structure.

Interesting feature

Indeed, both models have as common principle, the splitting of the input sequence in two period: the hindcast and the forecast period, each having a set of parametrizable inputs (features). This simple difference makes the prediction process easier in the construction of the input sequences.

Possible method to obtain a prediction

The use I thought was to give the model a context or hindcast period, where the LSTM cell would go through measures of dynamical features while having access to the streamflow measure, and then in the forecast period only giving the model the forecasted dynamical features, of course without the streamflow measure (no cheating). Using the architecture and specific properties of a typical LSTM model, we could ally both the memory construction in the hindcast phase and then the adaptation and update of the cell state during the forecast phase.

4.1 Description of the model

As mentioned before, using this method, we make use of having two different sets of features during the two periods onto which the cell will go.

After the LSTM cell went on the whole input sequence, the last N elements of the output list are extracted and pass by a dropout Layer (according to our config file) and finally a Linear Layer (simple FC network) to gives a sequence of N predictions. These predictions are then evaluated and the loss is computed.

This process is common to each sequence of inputs, that are fed following a batch logic, whose parameters are given in the config file (batch organized randomly).

Fig. 2 display a diagram of the main elements of this Sequential LSTM model.

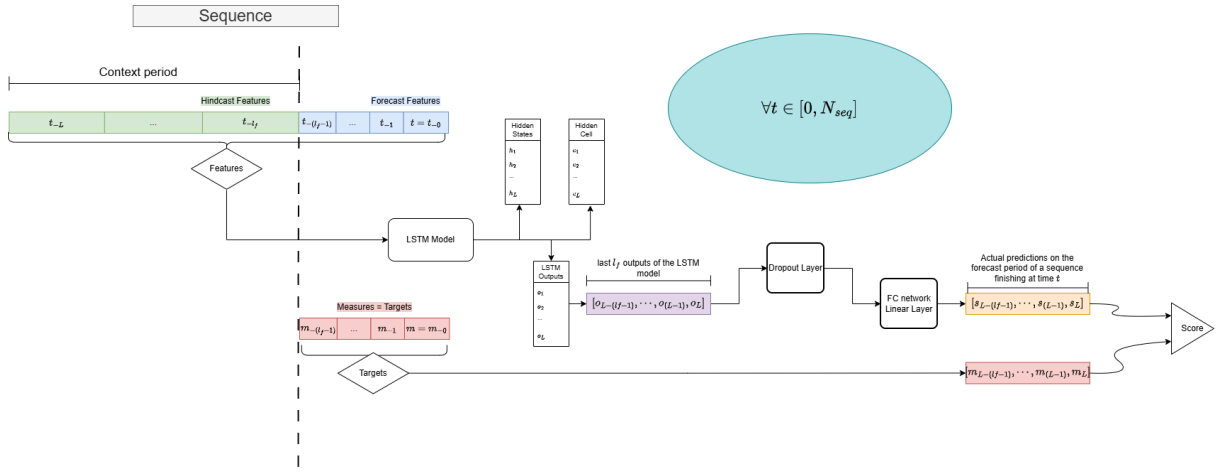


Fig. 2: Diagram of the implemented Hindcast-Forecast LSTM model

4.2 Questions

1. Only 1 lstm state and link feature-update for both hindcast and forecast part -> what if the structure of features is different? Need of same number of features for both part -> not very practical

    a. Model “handoff lstm forecast”: two diff lstm states (1 per sequence type), maybe more modularizable

2. Placement of the dropout layer (if we add one via the config) -> between lstm and head layers (current implementation) or do we prefer in the lstm layer, between the layers for example.

3. Computation of the loss: how to measure the model accuracy?

    a. Should we compute the loss over the forecast predictions only or do we want to – train a bit more the head layer, taking a state of the lstm at a moment of the sequence and producing a prediction, even during the hindcast period? Once more, the question of the feature organization in both periods is crucial for me -> maybe more feasible with a model type (“handoff hind/forecast”).

4.3 Results

Features:

- Tried several combos, currently trying with stations Bruchji, Fieschertal, Visp, Eggishorn and Jungfraujoch and rolling windows (sum over the last 6, 24h for example), not very efficient (mape >15-18). I think there are too many features.

With “basic features”, had obtained order of mape=12 for evaluation period and 13-15 in test mode.

- Not satisfying enough -> feedback on features, ideas of stations and other features
- RS inputs not yet tested -> should help the model by a large factor, but there too, need discussion about use in operational loop (specifically, at which time do we access which data to know at which part of the sequence do we feed this new data to our model

Maybe something like confi and a list of configs and then results or constatations by steps.

Only interesting steps presented, review test folders and clean a bit

5 Sequential LSTM using RS outputs

5.1 Preshow – promising results

Fig. 3 presents the score of the first try using the simple Hindcast-Forecast LSTM model with only RS outputs as features to train. The period of available data spanned from 1983 to 2024.

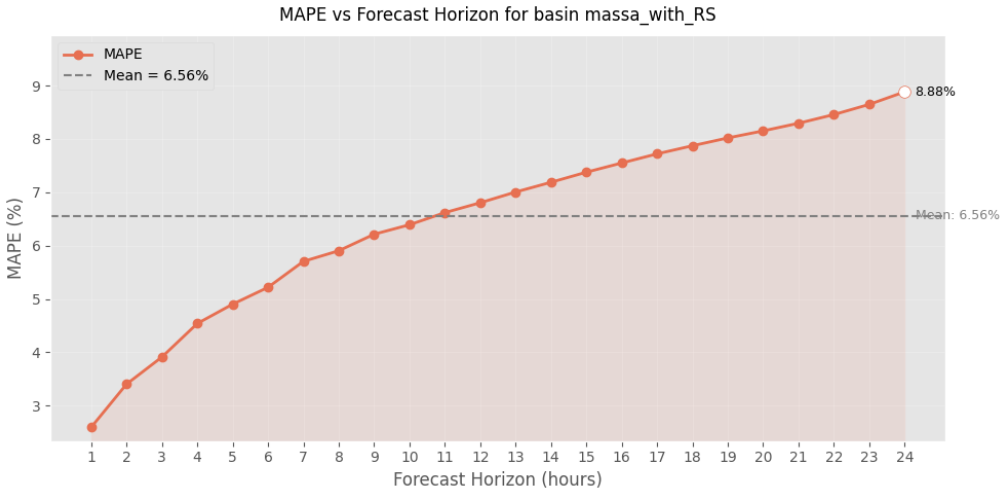


Fig. 3: Evolution of the MAPE score throughout the horizon of prediction.

6 NH – Opinion and what have been done

Python package (repo) with a “structured and practical” workspace including many “already coded” models.

NOTE: Precautions are to be taken with this code, i will describe here some mistakes or inconsistencies i found until now:

- Template for the config file is not usable, several names of variables not read by the config, clearly not up to date. I created a template personally with the main parameters (personal choice).
- The code is sometimes wrong or at least not usable in the current state. For example, the hindcast/forecast sequential LSTM model described below was modified, to better consider the two types of features and implement correctly the “counter” feature.
- The structure of the package is a bit too rigid, rendering the code not enough modularizable. To change a model architecture or one of its parameters, we often must go into the code and change the parameter ourselves. Some practical parameters are not implemented by the Config class (i think at the number of layers of our lstm right now).
  There are also other points such as the Scaler and the Loss where the code uses a self-implemented structure and classes instead of using the already existing, and working just fine, packages. (scikit-learn is perfect for Scaler and metrics already...)

Notes on NH

According to my experience, I would say that, depending on the idea we would like to implement, use NH repo is not really what could be the most optimized. What I mean is that if we only use 10% of the code and modify even only 10% of it, updating the package with our following implementations and fetching updates from their side could lead to some errors.

If their package is not usable only by creating a dataset loader class from our side, I personally think that for the future prosperity of our workflow, in the case we build a model that satisfy our need, we would better create a small python environment tuned for our use.

I have in mind that the model I tried were clearly not very advanced in the way they were implemented and the parameters must be changed in the code manually. Also concerning the dataset, we have anyways to create the class that will load the data, centre or transform it according to our expectations. The important part of the package could be the structure that it offers, but once again, we do not use most of the files. Even the dynamic learning rate had to be modified to be functional. I am ok with working with it but the simple fact that the sequences that we feed our model are “backward” is something we must consider as a “constraint” in our implementations.

6.1 Tools implemented – some of them at least

Data – selection, gathering:

RS export, csv with excel, careful that the name of the feature is in the first row, then align the data with the column date, that serves as the “index” of the csv

Data – analysis and inspection:

In a Jupyter notebook, basis analysis of the features: plot for visual comparison between measures and forecasts, basic statistical properties for math and rigorous comparison.
NAME: DB_Analysis.ipynb

Model and run parameters:

The configure a run and precise which parameter we want to use, put all the values in the config and save it in an yml formatted file (YAML). Template available in my repo.

To properly train and test our model, a template of a Jupyter notebook is also available. What this file need to run is the config file path.

Once the training and evaluation phase is finished, a function can provide the training and evaluation loss throughout the epochs and a second function allows to see the other important metrics and their evolution during training. (plot_training_curves, plot_validation_metrics)

Note: the logging and display during training has been changed (visual changes, to better capture the evolution of the losses and have a cleaner output).

Note2: For the test, it is necessary to provide the name of the model we want to test (model state saved every X epoch, parameterised with the config). For now, there is no option “SaveBest”, that would save the best model, according to a metric – validation loss by default, could be also the sum of important metrics (mse+nse+mape) – I want to implement it.

Once the model has finished the testing phase – form the test output file:

Functions:

- mape_array
  compute the MAPE metric for each horizon of prediction of our output (by default only the last element of our horizon is taken)

- plot_mape_horizon
  plot the MAPE metric in function of our horizon (increasing tendency)

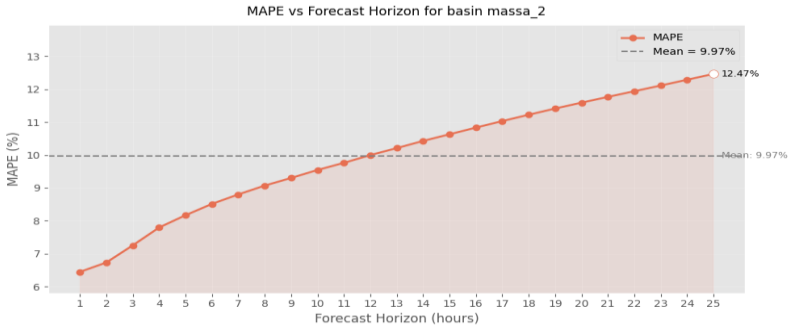


Fig. 4: Example of the evolution of a MAPE metric value for a complete horizon of prediction

- plot_forecast_24h
  interactive plot on the complete testing period, can choose which horizon we want to plot (int or list[int])

- plot_forecast_horizon_24h
  enter an issue date and it provides the forecast finishing at this date (entire horizon)

6.2 ML Typical workflow

Typical main steps:

1. Model motivation and selection
2. Data preparation
3. hyperparameters Tuning
4. Training phase and adaptation of the parameters
5. Testing phase when model seems good/promising
6. Saving the run folder

Challenge:

Each step is important and the order is important when consider what influence could have a modification onto other steps (possibly previously optimized).

Lslsls

s

References

Ref1

Ref2

Ref3

Cette étude a été réalisée par Flavien Kohler, cheffe stagiaire au bureau Hydrique Ingénieurs, et supervisée par Dr Frédéric Jordan, Directeur.

Fait au Mont-sur-Lausanne, en octobre-novembre 2025.

Hydrique Engineer HJ Sàrl

Dr Frédéric Jordan Dr Philippe Heller